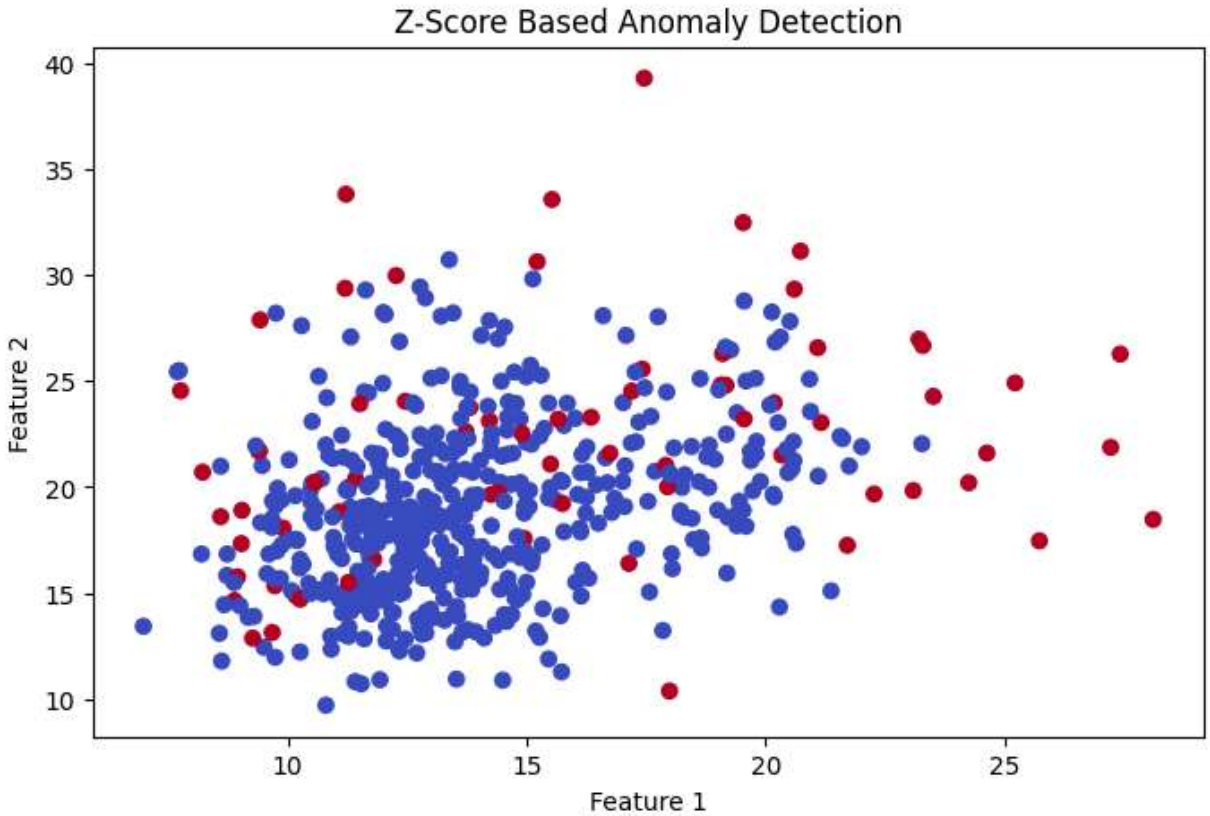
Practical 13: Anomaly Detection## Practical Information| Attribute | Details ||----------|---------|| **Practical Number** | 13 || **Title** | Anomaly Detection || **Duration** | 2 Hours || **Course Outcomes (COs)** | CO3, CO4, CO5 || **Description** | Detect anomalies and outliers in datasets using various techniques. |## Key Topics Covered- Statistical methods- Isolation Forest- LOF- One-class SVM- Autoencoder---## Instructions for Students1. **Complete all tasks** outlined in this practical2. **Write your code** in the cells provided below3. **Add comments** to explain your code4. **Include proper output** (prints, visualizations, etc.)5. **Submit your completed notebook** before the deadline---## Your Work Starts HereUse the cell(s) below to write your implementation:

```
# Practical 13 - Code 1: Statistical Anomaly Detection (Z-Score)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from scipy.stats import zscore

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)

# Calculate Z-scores
z_scores = np.abs(zscore(X))
```

```
# Set threshold
threshold = 3
outliers = (z_scores > threshold).any(axis=1)

print("Total Samples:", X.shape[0])
print("Number of Detected Outliers:", np.sum(outliers))

# Visualize using one feature
plt.figure(figsize=(8,5))
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=outliers, cmap='coolwarm')
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Z-Score Based Anomaly Detection")
plt.show()
```

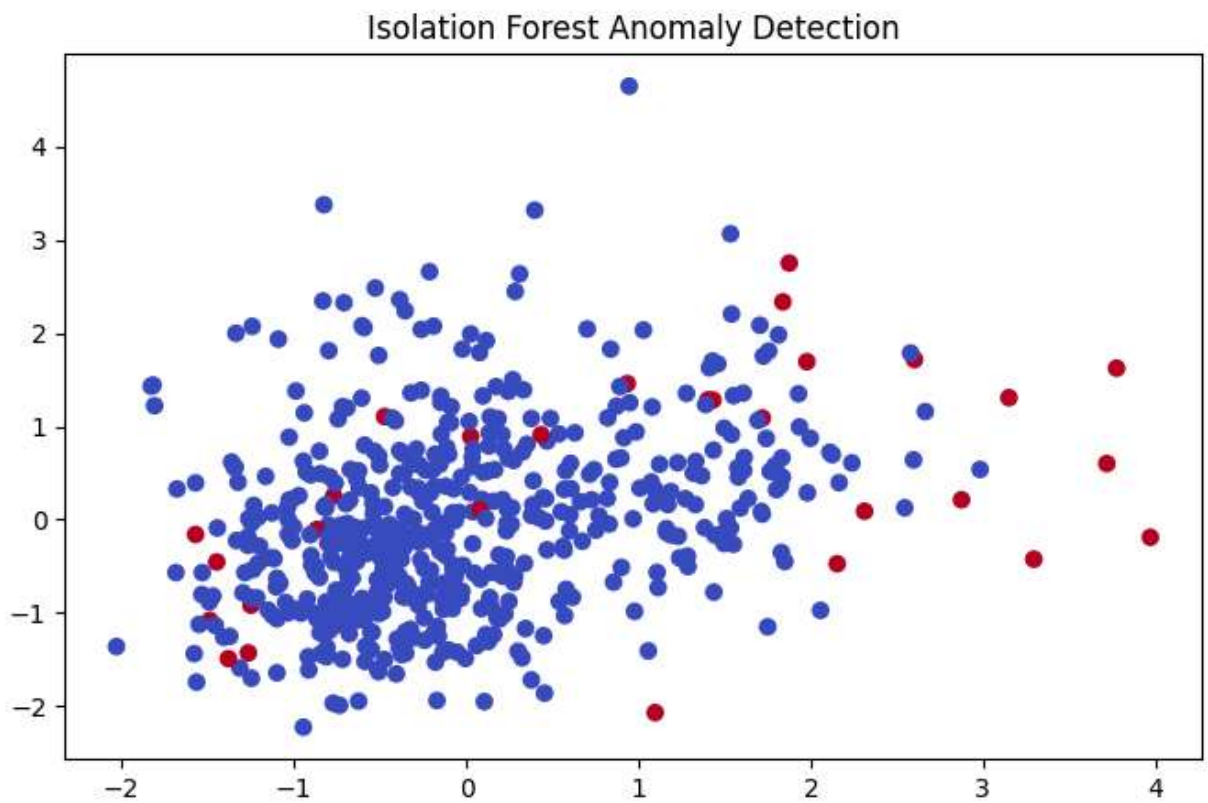
```
Total Samples: 569
Number of Detected Outliers: 74
```



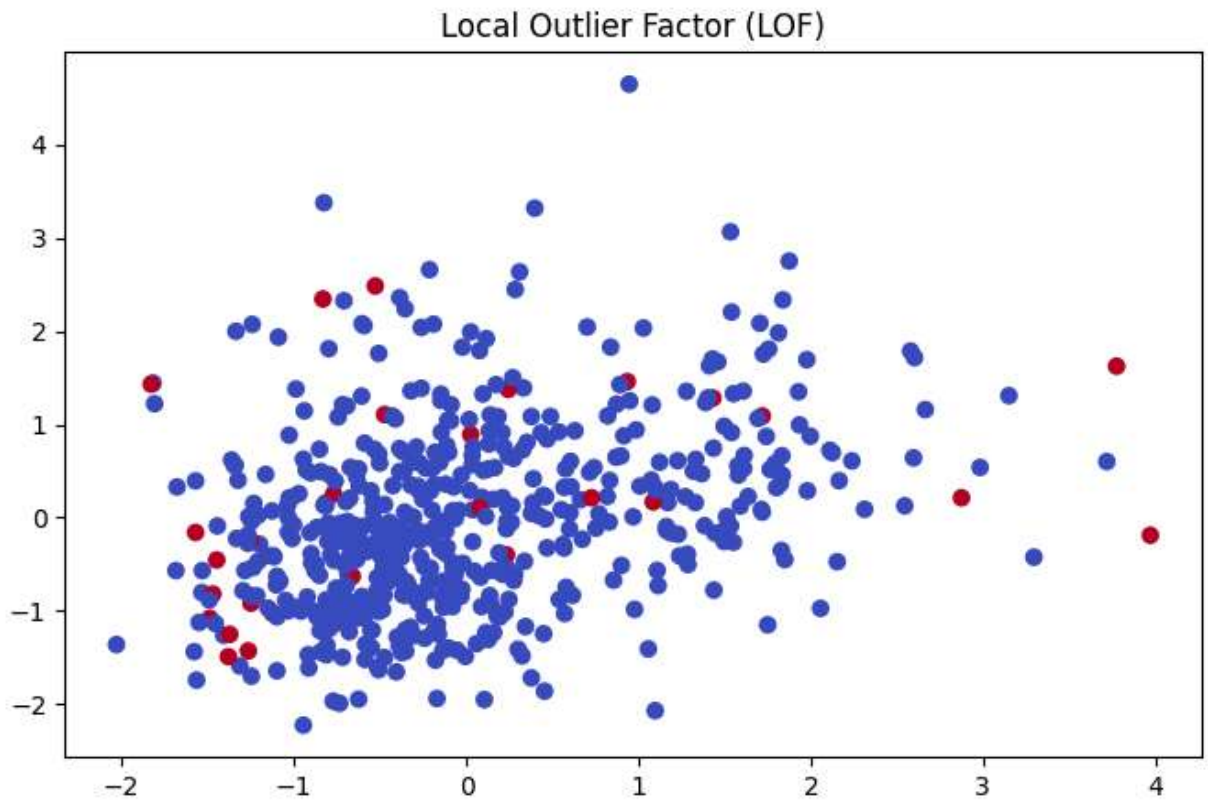

```
# Practical 13 - Code 2: Isolation Forest

from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Load dataset
```

```
X = load_breast_cancer().data

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train Isolation Forest
iso = IsolationForest(contamination=0.05, random_state=42)
y_pred = iso.fit_predict(X_scaled)

# -1 = anomaly, 1 = normal
anomalies = (y_pred == -1)

print("Total Samples:", len(X_scaled))
print("Number of Anomalies Detected:", sum(anomalies))

# Visualization (first two features)
plt.figure(figsize=(8,5))
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=anomalies, cmap='coolwarm')
plt.title("Isolation Forest Anomaly Detection")
plt.show()
```

```
Total Samples: 569
Number of Anomalies Detected: 29
```




```
# Practical 13 - Code 3: Local Outlier Factor (LOF)
```

```
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler

# Load dataset
X = load_breast_cancer().data

# Scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply LOF
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
y_pred = lof.fit_predict(X_scaled)

anomalies = (y_pred == -1)

print("Total Samples:", len(X_scaled))
print("Number of Anomalies Detected:", sum(anomalies))

# Plot
plt.figure(figsize=(8,5))
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=anomalies, cmap='coolwarm')
plt.title("Local Outlier Factor (LOF)")
plt.show()
```

```
Total Samples: 569
Number of Anomalies Detected: 29
```